

Banking Loan Risk Analysis Application: Step-by-Step Implementation Guide

Dataset: UCI *Default of Credit Card Clients* (30,000 rows) | **Stack:** Python, Pandas, FastAPI, PostgreSQL/MySQL, Streamlit | **Final submission:** Oct 11 (Milestone 1: Oct 5-7, Milestone 2: Oct 8-11, per the Telecom milestone slide; confirm the Banking dates with your organizers)

0. Read This First: Conflicts Between Your Three Documents

Issue	What the documents say	What to do
GenAI template	Submission_Guidelines.pdf mentions LLM, RAG, agents, DeepEval, RAGAS. The capstone deck is titled "Without AI" and Project 13 has no AI requirement.	Do not build RAG/agents. In README write "LLM / Embedding / Vector DB: Not applicable (rule-based analytics)". Skip Evaluation_Report; submit pytest results instead.
Folder structure	Guidelines: architecture/ design/ src/ docs/ tests/ presentation/. Deck: /app /etl /db /api /frontend /tests /docs /utils.	Use the Guidelines top level, and the deck's folders inside src/ (see Phase 0).
File names	Guidelines: Architecture_Diagram.pdf. Deck: 01_Architecture.pdf.	Use the Guidelines names (they are the formal ones). Mention it to organizers if unsure.
Deployment	Deck: Azure. Guidelines: local run is sufficient, Docker/Azure/CI-CD optional.	Run locally. Azure is a stretch goal only after everything else is done.
Repo name	Guidelines: <Name>_<Domain>, e.g. Veena_Telecom.	YourName_Banking (letters, digits, underscores only).

Marks (100): ETL 15, Data Quality 10, DB/SQL 15, Python 10, REST API 15, Analytics/KPIs 10, Dashboard 10, Testing 5, Architecture 5, Docs/Presentation 5. The APIs, ETL and SQL carry 45 marks, so protect time for them.

1. Plan at a Glance

Phase	Name	Milestone	Target
0	Setup and repo	M1	Oct 5
1	Data understanding and design decisions	M1	Oct 5
2	Database schema	M1	Oct 5-6
3	Basic ETL pipeline	M1	Oct 6
4	SQL KPI queries	M1	Oct 6
5	REST API with CRUD	M1	Oct 6-7

6	Basic dashboard + M1 checkpoint	M1	Oct 7
7	Incremental ETL, reconciliation, data-quality framework	M2	Oct 8
8	Risk bands (business rules)	M2	Oct 8
9	Advanced API: pagination, filters, sort, export, RBAC	M2	Oct 9
10	DB optimization and aggregate tables	M2	Oct 9
11	Testing, logging, scheduling	M2	Oct 10
12	Enhanced UI	M2	Oct 10
13	Documentation, deliverables, presentation, submission	M2	Oct 10-11

Rule for every phase: **do not start it until its "Before you start" list is fully ticked.** Commit to Git at the end of each phase.

MILESTONE 1: Core Solution (about 60%)

Phase 0: Setup and Repository

Before you start: nothing.

Tasks

1. Install Python 3.10+, PostgreSQL (or MySQL), Git, VS Code. Create a virtual environment.
2. Install: pandas openpyxl sqlalchemy psycopg2-binary fastapi uvicorn pydantic streamlit requests pytest python-jose passlib[bcrypt] apscheduler pyyaml plotly python-dotenv. Freeze to requirements.txt.
3. Create the GitHub repo YourName_Banking and this structure:

```

YourName_Banking/
├── architecture/           Architecture_Diagram.pdf
├── design/                 Design_Document.pdf
├── src/
│   ├── app/               main application / config
│   ├── api/               FastAPI routers, schemas, auth
│   ├── etl/               extract.py transform.py validate.py load.py pipeline.py
│   ├── db/                schema.sql, indexes.sql, kpi_queries.sql, seed
│   ├── frontend/          Streamlit pages
│   ├── utils/              logger, config loader, helpers
│   ├── config/             risk_rules.yaml, dq_rules.yaml
│   └── main.py
├── data/ (raw/, processed/, sample_data/)
├── docs/                   API_Documentation.pdf, Dataset_Details.pdf
├── tests/
├── presentation/           Project_Presentation.pptx
├── requirements.txt
├── README.md
├── .gitignore
└── .env.example

```

4. .gitignore: .env, venv/, __pycache__/, large raw files if needed. **Never commit passwords or secrets.** Use .env plus a committed .env.example.
5. Create the database bank risk and confirm Python can connect.

Done when: uvicorn starts a "hello" endpoint, DB connection test passes, first commit is pushed.

Phase 1: Data Understanding and Design Decisions

Before you start: Phase 0 done; dataset downloaded into `data/raw/` from the UCI link.

Tasks

1. Explore in a notebook (`docs/eda.ipynb` optional): shape, dtypes, nulls, duplicates, distributions.
2. Note the known dataset quirks and decide a rule for each (these become your data-quality checks and a talking point in the demo):
 - The Excel file has a header on the **second** row; the target column is `default payment next month`.
 - Columns go `PAY_0`, `PAY_2`, ... `PAY_6` (there is no `PAY_1`). Rename `PAY_0` to `PAY_1` internally and document it.
 - `EDUCATION` is documented as 1-4, but 0, 5, 6 appear. `MARRIAGE` documented 1-3, but 0 appears. Map them to "Other/Unknown".
 - `PAY_x` values: -1 = paid duly, 1-9 = months delayed; -2 and 0 are undocumented (treat as "no consumption" and "revolving credit"). Document this.
 - `BILL_AMT` can be negative (credit balance). Keep it but flag it.
 - Month mapping: index 1 = September 2005 ... index 6 = April 2005. `PAY_AMT1` is the payment made in September toward the August bill.
3. Write the **design decisions** in a running file (`design/notes.md`): why PostgreSQL, why FastAPI, wide-to-long reshaping of bills and payments, how rejects are handled. This feeds the Design Document later.
4. Draft the risk rules (see Phase 8) on paper now so the schema stores what you need.

Done when: you have a list of quirks and the cleaning rule for each, and a one-page data dictionary.

Phase 2: Database Schema

Before you start: Phase 1 data dictionary complete.

Tasks. Write `src/db/schema.sql` (3NF, reshaping the wide file into long tables):

Table	Key columns
customers	customer_id PK, sex, education, marriage, age
credit_profiles	customer_id PK/FK, limit_bal, default_next_month, risk_score, risk_band (filled in Phase 8)
bills	bill_id PK, customer_id FK, bill_month (date), bill_amt, UNIQUE(customer_id, bill_month)
payments	payment_id PK, customer_id FK, pay_month (date), pay_amt, repay_status, UNIQUE(customer_id, pay_month)
etl_run_log	run_id, file_name, file_hash, started_at, ended_at, rows_read, rows_loaded, rows_rejected, status
rejected_records	id, run_id, customer_id, reason, raw_row

users user_id, username, password_hash, role (analyst/admin)

Add CHECK constraints (`limit_bal > 0`, `age BETWEEN 18 AND 100`, `sex IN (1,2)`). Basic indexes now; more in Phase 10. Store `bill_month` / `pay_month` as real dates so date-range queries are possible.

Done when: schema runs cleanly on an empty DB; an ER diagram is exported (it goes into the Design Document).

Phase 3: Basic ETL Pipeline

Before you start: Phase 2 tables exist; raw file present.

Tasks. Build reusable modules (not one script):

1. `extract.py`: read CSV/XLS with correct header row; return DataFrame.
2. `transform.py`: rename columns, map invalid EDUCATION/MARRIAGE, convert wide to long (`melt`) for bills and payments, build month dates, enforce dtypes.
3. `validate.py`: configurable rules from `config/dq_rules.yaml` (ranges, allowed values, nulls). Failing rows go to `rejected_records` with a reason; they do not crash the run.
4. `load.py`: load into the four tables with SQLAlchemy, in batches, inside a transaction.
5. `pipeline.py`: orchestrates extract, transform, validate, load; wrap each stage in try/except and log with Python `logging` (file + console).
6. Write a row to `etl_run_log` for each run.

Done when: one command (`python -m src.etl.pipeline`) loads all 30,000 customers, about 180,000 bill rows and about 180,000 payment rows; counts match the source (`read = loaded + rejected`).

Phase 4: SQL KPI Queries

Before you start: data loaded (Phase 3).

Tasks. Put these in `src/db/kpi_queries.sql` (JOINS and GROUP BY, window functions where useful):

- Total and average credit exposure (`limit_bal`), by segment (sex, education, marriage, age group).
- Average utilization = `bill / limit`, by segment and by month.
- Default rate by segment (Q1: which segments have higher payment risk).
- Monthly trend of total bills and total payments (Q4).
- Payment-to-bill ratio by month; count of customers with repayment delay of 2+ months.
- Top N customers by exposure/delay (review candidates).
- Limit-vs-behavior comparison (limit bands vs average delay/payment) (Q2).

Done when: every query runs, returns sensible output, and each answers one of the four business questions. Save sample outputs for the README.

Phase 5: REST API with CRUD (Basic)

Before you start: schema + data loaded; KPI queries validated.

Tasks (FastAPI, `src/api/`, with Pydantic schemas and a service/repository layer):

- `POST /customers`, `GET /customers/{id}`, `PUT /customers/{id}`, `DELETE /customers/{id}`, `GET /customers`.
- Validation (Pydantic: age, limit, enums) and proper errors: 404 422 409 (duplicate) 500 handler

validation (y, gender, age, limit, status), and proper errors: 101, 102, 103 (duplicate), 500 (handler).

- Analytics endpoints wrapping Phase 4: GET /analytics/exposure, /analytics/segments, /analytics/monthly-trends.
- GET /health. Swagger is automatic at /docs; add descriptions and examples to every endpoint.

Done when: every endpoint works from Swagger and returns correct data; Swagger screenshots saved.

Phase 6: Basic Dashboard and Milestone 1 Checkpoint

Before you start: API endpoints from Phase 5 are working.

Tasks (Streamlit, calling the API, not the DB directly, to show the architecture):

- KPI cards: total customers, total exposure, average utilization, default rate.
- Charts: exposure by segment, default rate by segment, monthly bills vs payments trend.
- Sidebar filters (sex, education, marriage, age range).
- A customer lookup/search view.

M1 checkpoint (tick all):

- ☐ ETL runs end to end with logging
- ☐ Data in PostgreSQL/MySQL, KPI SQL done
- ☐ CRUD APIs + Swagger
- ☐ Basic dashboard
- ☐ Commit and push; try a 2-minute demo of "top high-exposure segments"

MILESTONE 2: Advanced Features and Final Deliverables (about 40%)

Phase 7: Incremental ETL, Reconciliation, Data-Quality Framework

Before you start: M1 checkpoint ticked; basic ETL reliable.

Tasks

1. **Create incremental input:** split the dataset into batch files (for example batch_1.csv ... batch_4.csv), and make one later batch contain updated rows for existing IDs plus a few deliberately bad rows (duplicates, negative limits, age 200, invalid codes) so you can demonstrate the checks.
2. **Incremental logic:** watch data/incoming/; skip files whose hash is already in etl_run_log; **upsert** (INSERT ... ON CONFLICT DO UPDATE) so reruns are idempotent; move processed files to data/processed/.
3. **Reconciliation:** after each run, compare source rows vs loaded + rejected, and checksums (sum of limit_bal, sum of bills) between file and DB; store the result and flag mismatches. Expose it in GET /etl/runs/{id}/reconciliation.
4. **Reusable DQ checks** in a dq/ module driven by dq_rules.yaml: duplicates, invalid ranges, inconsistent records (for example a payment status of "paid duly" with a large unpaid bill, or a negative age), missing values. Output a DQ report table dq_issues.

Done when: running the same file twice loads nothing new; a dirty batch produces rejects and a reconciliation report.

Phase 8: Rule-Based Risk Bands

Before you start: clean, incrementally loaded data.

Tasks. Put thresholds in `config/risk_rules.yaml` (configurable and documented, as the project requires). Example starting rules, which you should tune and then justify in the Design Document:

- Derived metrics per customer: `max_delay` (max of `PAY_1..PAY_6`), `delay_months` (count of months with status ≥ 1), `utilization` = `BILL_AMT1` / `LIMIT_BAL`, `payment_ratio` = `PAY_AMT1` / `BILL_AMT2`.
- **High** if `PAY_1` ≥ 2 or (`utilization` ≥ 0.9 and `payment_ratio` < 0.2) or `delay_months` ≥ 4 .
- **Medium** if `PAY_1` $= 1$ or `utilization` ≥ 0.6 or `delay_months` ≥ 2 .
- **Low** otherwise.
- Store `risk_band`, the score and a short `risk_reason` per customer; recompute after each ETL run.
- **Validation trick:** compare actual default rate per band. A good ruleset shows High > Medium > Low; show this table in the demo.
- Review queue: `GET /risk/review-queue` returns High-band customers with reasons (answers Q3).

Done when: every customer has a band, rules are changeable via YAML without code edits, and band default rates are in the right order.

Phase 9: Advanced API: Pagination, Filtering, Sorting, Export, RBAC

Before you start: risk bands populated; basic API stable.

Tasks

- `GET /customers?`
`page=1&page_size=50&sort_by=limit_bal&order=desc&risk_band=High&education=2&age_min=25&age_max=40&limit_min=...&month_from=2005-04&month_to=2005-09`. Return `{items, page, page_size, total, total_pages}`.
- Date-range endpoints for bills and payments (`/bills?from=&to=`).
- Export: `GET /export/customers.csv` with the same filters (stream it); optionally `/export/risk-report.csv`.
- **Auth + RBAC:** `POST /auth/login` returns a JWT; a dependency checks the role. **Analyst:** read, analytics, export. **Admin:** all of that plus create/update/delete, trigger ETL (`POST /etl/run`), edit rules, manage users. Seed one analyst and one admin (hashed passwords).
- Return 401/403 correctly and document the role of each endpoint.

Done when: an analyst token gets 403 on DELETE and an admin token succeeds; pagination and filters verified in Swagger.

Phase 10: Database Optimization and Aggregate Tables

Before you start: the filters and queries from Phase 9 exist (you optimize what you can measure).

Tasks

1. Run `EXPLAIN ANALYZE` on your slowest queries (before).
2. Add indexes in `db/indexes.sql`: `bills(customer_id, bill_month)`, `payments(customer_id, pay_month)`, `credit_profiles(risk_band)`, `customers(education, marriage, sex)`, plus composite indexes matching your filters.
3. Create aggregate/reporting tables, refreshed after each ETL run: `agg_segment_summary` (segment,

4. Re-run `EXPLAIN ANALYZE` (after) and record the timings for the Design Document.

Done when: a documented before/after improvement exists.

Phase 11: Automated Tests, Logging, Scheduling

Before you start: features in Phases 7-10 working (tests lock them in).

Tasks

- **pytest** in `tests/`: ETL (transform mapping, wide-to-long counts, validation rejects, idempotent rerun), business rules (each band boundary), SQL queries (against a small test DB with known fixtures), APIs (FastAPI `TestClient`: CRUD, pagination, 401/403, export). Aim for roughly 30+ meaningful tests; save `pytest` output as evidence.
- **Logging:** rotating file handler, levels, run IDs in messages; API request logging.
- **Scheduling:** `APScheduler` (or `cron`/`Airflow` concepts) to run the incremental pipeline daily (for the demo use a short interval); log each run in `etl_run_log`, plus failure handling (retry once, log an error).

Done when: `pytest` passes fully; the scheduler picks up a new file dropped into `data/incoming/` on its own.

Phase 12: Enhanced UI

Before you start: RBAC and advanced APIs working.

Tasks. Extend `Streamlit`: login page (token stored in session state); paginated, filterable, sortable customer table; risk-band distribution chart; review-queue page; CSV download button; ETL run history and reconciliation status page (admin only); show/hide pages by role.

Done when: an analyst and an admin see different options in the UI and every screen uses the API.

Phase 13: Documentation, Deliverables, Presentation, Submission

Before you start: all features done and tested. Freeze code except for bug fixes.

Deliverables checklist (use the Guidelines naming):

Deliverable	Contents
<code>architecture/Architecture_Diagram.pdf</code>	Dataset, then ETL (extract, validate, transform, load, reconciliation), then DB (tables + aggregates), then FastAPI (auth/RBAC), then <code>Streamlit</code> ; scheduler/logging on the side. Use <code>draw.io</code> ; export PDF.
<code>design/Design_Document.pdf</code>	Problem statement, architecture, tech choices and justification, schema/ER diagram, ETL design, risk-rule rationale, DQ strategy, security/RBAC, optimization results, trade-offs and limitations (for example, rules vs ML, local vs cloud).
<code>src/</code>	Complete, modular, runnable code.
<code>README.md</code>	Overview, problem, objective, solution summary, tech

stack (LLM/RAG: N/A), install, .env setup, DB setup and schema, ETL run steps, start commands, 3-5 sample requests and responses.

docs/API_Documentation.pdf

Per endpoint: URL, method, request, response, sample request/response, roles. Export from Swagger/ReDoc and complete it.

docs/Dataset_Details.pdf

Name, source, link, format, key fields, purpose, preprocessing done.

presentation/Project_Presentation.pptx

10 min: problem, business need, solution, architecture, data flow, tech stack, demo, results, challenges and learnings.

data/sample_data/

Small sample CSV (about 100 rows).

Demo video (optional, 5-7 min MP4)

Unlisted link.

Local run commands (to put in README):

```
uvicorn src.main:app --reload
streamlit run src/frontend/app.py
```

Demo script (8 min demo + 2 min Q&A): (1) drop a new file and watch incremental ETL + reconciliation; (2) show DQ rejects; (3) dashboard: segments and risk bands; (4) API in Swagger: filtered, paginated call; (5) analyst vs admin permissions; (6) tests passing; (7) answer the four business questions with real numbers. Rehearse twice.

Final submission checklist

- ☐ Fresh clone runs by following only the README
- ☐ No secrets committed (.env ignored, .env.example present)
- ☐ All files named per convention, no spaces
- ☐ APIs tested; sample data included
- ☐ Repo is accessible to the organizers
- ☐ Share: repo URL, demo video URL (optional), team details (names, ID, email)

Appendix: Quick Answers to the Four Business Questions (what your demo must show)

1. **Higher-risk segments:** default rate and High-band share by sex, education, marriage, age group.
2. **Limits vs behavior:** limit bands vs utilization, delay, payment ratio.
3. **Records needing review:** the High-band review queue with reasons.
4. **Trends:** monthly total bills, payments and utilization (Apr-Sep 2005).

Appendix: If Time Runs Short, Cut in This Order

Keep ETL, DB, APIs, tests, docs first (about 80 of 100 marks). Cut: Azure deployment, demo video, Airflow (use APScheduler), fancy UI styling.

